

Tableview Refactoring Plan

Technical title: `QTableView + PandasTableModel + QSortFilterProxyModel Refactor Plan`

Document file: `Design Assets/260630 QTableView Refactor Plan KD.md`

Project: RFT 2026 — Analysis View

Scope: Tabular frame + graphical frame (shared display data). Data loaders and `show_dataframe_table_dialog` remain on `QTableWidget`.

Date: 2026-06-30

Follow this plan as explicitly as possible. Implementation proceeds **phase by phase**; request **full code for each numbered item** (e.g. "Phase 0, item 3") when ready to build. Items **1–55** are listed in §6.

1. Background

Current arrangement (`QTableWidget`)

`view.df` → `create_table_view_frame` → copy every cell into `QTableWidget` → full rebuild on change

- Row 0 holds `UnitsComboBox` widgets; data rows are `QTableWidgetItem` copies.
- Column tick/untick and unit changes trigger `tabular_frame.update_table()` — destroy widget tree and rebuild.
- Works for small tables; becomes sluggish with hundreds/thousands of rows and frequent Excel-style filtering/sorting.

Target arrangement (Model / View / Proxy)

```
view.df (source, column-selected)
  ↓
PandasTableModel          ← reads df on demand, formats for display
  ↓
ViewSortFilterProxyModel ← row filter + sort (no full rebuild)
  ↓
QTableView (tabular)      GraphicalFrame (reads same visible rows)
```

Both `TabularFrame` and `GraphicalFrame` subscribe to one **ViewDisplayController** that owns the model + proxy and answers: *which rows are visible?*

Conceptual comparison

Aspect	Current <code>QTableWidget</code>	Target <code>QTableView</code> + model + proxy
Where data lives	<code>view.df</code> + copied into items	Primarily <code>view.df</code> ; model reads on demand
Filter / sort	Rebuild or manual row hide	Proxy maps rows; <code>invalidateFilter()</code>
Performance	Slow on large rebuilds	Only visible cells queried

Aspect	Current <code>QTableWidget</code>	Target <code>QTableView</code> + model + proxy
Units row	Row 0 <code>setCellWidget</code>	Toolbar above table (not a data row)
Sortable headers	Custom code needed	<code>QTableView.setSortingEnabled(True)</code> on proxy

2. Architecture

Data layers

Layer	Responsibility
<code>view.df</code>	Canonical working dataframe (column selection + excess pressure column applied)
<code>view.column_specs</code>	Display units and quantity keys aligned with <code>view.df</code> columns
<code>view.row_filters</code> <i>(new field)</i>	Persisted filter rules for save/load
<code>PandasTableModel</code>	Adapter: answers <code>data(row, col)</code> with formatted display strings
<code>ViewSortFilterProxyModel</code>	Hides/reorders rows without mutating <code>view.df</code>
<code>ViewDisplayController</code>	Single owner of model + proxy; emits <code>display_changed</code>
<code>GraphicalFrame</code>	Plots <code>get_visible_dataframe()</code> , not raw <code>view.df</code>

Signal flow (orchestrator)

```
ViewSidebar.view_df_changed
  → on_view_df_change
    → build_view_df_and_col_specs_from_column_selection
    → controller.set_view_data(df, column_specs)
    → mark_modified()

TabularFrame.column_unit_change
  → _on_column_unit_change
    → on_column_unit_change(view, ...)
    → controller.refresh_formatting()
    → mark_modified()

Filter UI / proxy
  → on_row_filter_change
    → controller.set_filters(...)
    → display_changed
      → graphical_frame.update_from_visible_data(...)
```

3. Function map

A. Shared display formatting (extract from `global_functions.py`)

Keep unit conversion / rounding in one place. Data-loader dialogs keep `QTableWidget`; they may later call the same formatters.

Function	Description	Proposed file	Replaces
<code>format_normalised_cell_value(value, quantity_key, output_unit) -> str</code>	Convert one normalized df value to user-unit display text (handles NA / non-numeric).	<code>ui/analysis_view/model/view_table_formatting.py</code>	Inner loop in <code>create_dataframe_table</code> → <code>update_column_values</code> (<code>global_functions.py</code> ~148–158)
<code>apply_decimal_formatting(display_str, *, round_enabled, decimal_places) -> str</code>	Apply optional rounding to a display string.	same file	<code>apply_rounding_to_column</code> + <code>round_str_to_decimal_points</code> path in <code>create_table_view_frame</code> → <code>refresh_column</code> (<code>global_functions.py</code> ~245–256)
<code>format_cell_for_table(value, column_spec, output_unit, decimal_settings) -> str</code>	Single entry point: unit convert + round for one cell.	same file	Combined per-cell work in <code>update_column_values</code> + <code>apply_rounding_to_column</code>

Note: `round_str_to_decimal_points` in `global_functions.py` **stays** (used by loaders). Analysis view calls the new wrappers above.

B. Filter / view data model

Function / type	Description	Proposed file	Replaces
<code>FilterSpec</code> (dataclass)	One rule: column name, operator (<code>></code> , <code><</code> , <code>==</code> , <code>between</code> , <code>in</code>), value(s).	<code>ui/analysis_view/model/filter_spec.py</code>	Nothing (new)
<code>row_filter_mask(df, filter_specs, column_specs) -> pd.Series[bool]</code>	Pure pandas AND mask across all active filters.	<code>ui/analysis_view/model/analysis_view_data_manager.py</code>	Future ad-hoc filter code
<code>on_row_filter_change(view, filter_specs) -> None</code>	Store filters on <code>AnalysisView</code> , ready for proxy invalidation.	same file	Nothing (new)
<code>get_visible_row_indices(proxy) -> list[int]</code>	Map proxy row 0..n-1 to source df row indices (for graph).	same file	Nothing (new)
<code>get_visible_dataframe(view, proxy) -> pd.DataFrame</code>	<code>view.df.iloc[visible_indices]</code> — single source for table + plot.	same file	Implicit “whole df” reads in future graph code
<code>get_plot_series(view, visible_df) -> dict</code>	Extract depth / pressure / excess columns for plotting (canonical names).	same file	Nothing (new)

Model field to add: `AnalysisView.row_filters: list[FilterSpec]` in `project/models.py`.

Unchanged: `build_view_df_and_col_specs_from_column_selection`,
`insert_excess_pressure_column`, `on_column_unit_change`.

C. Qt model / proxy layer

Function / class	Description	Proposed file	Replaces
<code>PandasTableModel</code>	<code>QAbstractTableModel</code> reading <code>view.df</code> ; <code>data()</code> calls <code>format_cell_for_table</code> .	<code>ui/analysis_view/model/pandas_table_model.py</code>	<code>create_dataframe_table</code> for analysis view only (<code>global_functions.py</code> 127–174)
<code>PandasTableModel.set_dataframe(df, column_specs, project, decimal_settings)</code>	Swap underlying data and emit <code>modelReset</code> (column tick/untick).	same class	Full <code>tabular_frame.update_table()</code> rebuild
<code>PandasTableModel.set_column_unit(col, unit)</code>	Update display unit for one column; emit <code>dataChanged</code> for that column.	same class	<code>update_column_values(col)</code> in <code>create_dataframe_table</code>
<code>PandasTableModel.set_decimal_settings(...)</code>	Reformat all numeric cells without structural rebuild.	same class	<code>refresh_all_columns</code> in <code>create_table_view_frame</code> (<code>global_functions.py</code> 258–260)
<code>ViewSortFilterProxyModel</code>	<code>QSortFilterProxyModel</code> subclass; <code>filterAcceptsRow</code> uses <code>row_filter_mask</code> .	<code>ui/analysis_view/model/view_filter_proxy_model.py</code>	Nothing (new)
<code>ViewSortFilterProxyModel.set_filters(filter_specs)</code>	Update rules and <code>invalidateFilter()</code> .	same class	Nothing (new)
<code>ViewSortFilterProxyModel.source_row_at(proxy_row) -> int</code>	Thin wrapper over <code>mapToSource</code> for graph code.	same class	Nothing (new)

D. Display coordinator (shared table + graph)

Function / class	Description	Proposed file	Replaces
<code>ViewDisplayController</code>	Owns <code>PandasTableModel</code> , <code>ViewSortFilterProxyModel</code> , decimal settings.	<code>ui/analysis_view/model/view_display_controller.py</code>	Ad-hoc model/proxy creation inside <code>tabular_frame</code>
<code>ViewDisplayController.set_view_data(df, column_specs)</code>	Push new df/specs into source model after column selection.	same class	<code>tabular_frame.update_table()</code> on column change

Function / class	Description	Proposed file	Replaces
<code>ViewDisplayController.set_filters(filter_specs)</code>	Push filters into proxy.	same class	Nothing (new)
<code>ViewDisplayController.get_visible_dataframe() -> pd.DataFrame</code>	Delegates to <code>get_visible_dataframe(view, proxy)</code> .	same class	Nothing (new)
<code>ViewDisplayController.refresh_formatting()</code>	Unit or decimal change → <code>dataChanged</code> only.	same class	<code>tabular_frame.update_table()</code> on unit change; <code>refresh_column</code> / <code>refresh_all_columns</code> in <code>create_table_view_frame</code>

Signal: `display_changed` — emitted when filter, sort, df, or format changes; tabular and graphical frames listen.

E. Tabular UI

Function	Description	Proposed file	Replaces
<code>_build_table_view_on_create()</code>	Create <code>QTableView</code> , attach proxy, enable header sort — once in <code>__init__</code> .	<code>ui/analysis_view/ui/tabular_frame.py</code>	<code>_build_ui</code> + <code>_create_table</code> destroy/recreate pattern
<code>_build_display_toolbar()</code>	Units combos per column + decimal checkbox/spin (not data row 0).	<code>ui/analysis_view/ui/tabular_display_toolbar.py</code>	<code>widgets_frame</code> in <code>create_table_view_frame</code> (<code>global_functions.py</code> 223–270)
<code>_sync_units_toolbar_from_specs()</code>	Set combo texts from <code>view.column_specs</code> .	<code>table_display_toolbar.py</code> or <code>tabular_frame.py</code>	<code>_update_table_units_combo_from_specs</code>
<code>_on_toolbar_unit_changed(col, unit)</code>	Emit <code>column_unit_change</code> + <code>controller.refresh_formatting()</code> .	<code>tabular_frame.py</code>	<code>_link_table_units_combo_with_signal</code> + units wiring in <code>create_table_view_frame._connect_signals</code>
<code>_on_decimal_settings_changed()</code>	Forward to <code>controller.refresh_formatting()</code> .	<code>tabular_frame.py</code>	Decimal spin/checkbox connections in <code>create_table_view_frame</code>
<code>bind_display_controller(controller)</code>	Connect view to shared controller.	<code>tabular_frame.py</code>	<code>create_table_view_frame(...)</code> call in <code>_create_table</code>
<code>set_view_data(df, column_specs)</code>	<code>controller.set_view_data(...)</code> — no widget destroy.	<code>tabular_frame.py</code>	<code>update_table()</code>

Function	Description	Proposed file	Replaces
<code>set_row_filters(filter_specs)</code>	<code>controller.set_filters(...)</code>	<code>tabular_frame.py</code>	Nothing (new)

Removed after refactor (tabular only):

Removed	Was in
<code>update_table()</code>	<code>tabular_frame.py</code>
<code>_create_table()</code>	<code>tabular_frame.py</code>
<code>_update_table_units_combo_from_specs()</code>	<code>tabular_frame.py</code>
<code>_link_table_units_combo_with_signal()</code>	<code>tabular_frame.py</code>

F. Filter UI (Excel-style — Phase 4)

Function / class	Description	Proposed file	Replaces
<code>FilterableHeaderView</code>	Header click opens filter popup for that column.	<code>ui/analysis_view/ui/filterable_header_view.py</code>	Nothing (new)
<code>ColumnFilterPopup</code>	Operator combo + value field + optional unique-value checklist.	<code>ui/analysis_view/ui/column_filter_popup.py</code>	Nothing (new)
<code>_on_filter_applied(col, filter_spec)</code>	Tabular handler → orchestrator → <code>on_row_filter_change</code> .	<code>tabular_frame.py</code>	Nothing (new)

Sortable headers: `QTableView.setSortingEnabled(True)` on the proxy — no custom sort code required.

G. Graphical frame

Function	Description	Proposed file	Replaces
<code>bind_display_controller(controller)</code>	Hold reference; connect <code>display_changed</code> .	<code>ui/analysis_view/ui/graphical_frame.py</code>	Nothing
<code>update_from_visible_data(visible_df, column_specs)</code>	Public refresh when table filter/sort/df changes.	same file	Empty <code>_build_ui</code>
<code>_render_plot(visible_df)</code>	Draw depth vs pressure (library TBD).	same file	Nothing (new)
<code>_clear_plot()</code>	Clear when no visible rows.	same file	Nothing (new)

Rule: Graph uses `controller.get_visible_dataframe()`, not `view.df` directly.

Fix required: `analysis_view.py` currently assigns `graphical_frame` as a local variable; refactor must use `self.graphical_frame`.

H. Orchestrator (`analysis_view.py`)

Function	Description	Proposed file	Replaces
----------	-------------	---------------	----------

Function	Description	Proposed file	Replaces
<code>_init_display_controller()</code>	Create one <code>ViewDisplayController</code> for tabular + graph.	<code>analysis_view.py</code>	Implicit per-frame df binding
<code>_on_display_changed()</code>	<code>graphical_frame.update_from_visible_data(..)</code> .	<code>analysis_view.py</code>	Nothing (new)
<code>on_view_df_change()</code> (extend)	After updating <code>view.df</code> / specs → <code>controller.set_view_data(...)</code> .	<code>analysis_view.py</code>	<code>self.tabular_frame.update_table()</code>
<code>_on_column_unit_change()</code> (extend)	After <code>on_column_unit_change</code> → <code>controller.refresh_formatting()</code> .	<code>analysis_view.py</code>	Full table rebuild on unit change
<code>_on_row_filter_change()</code> (new)	Filter UI → <code>on_row_filter_change</code> + <code>controller.set_filters</code> .	<code>analysis_view.py</code>	Nothing (new)

4. What stays untouched

Area	Stays as-is
<code>dialog_data_loader_project.py</code>	<code>QTableWidget</code> preview/mapping
<code>dialog_data_loader_analysis.py</code>	Own <code>_update_table</code> / <code>_update_table_values</code>
<code>create_dataframe_table</code> , <code>create_table_view_frame</code>	Used by <code>show_dataframe_table_dialog</code> (tree “view dataset”)
<code>create_log_table</code> , <code>show_log_table</code>	Log viewer
<code>view_sidebar.py</code>	Column tick/untick (structural column filter)
<code>build_view_df_and_col_specs_from_column_selection</code>	Column subset + excess pressure column
Excess pressure calculation	Future work (stub remains)

5. Proposed folder layout

```

ui/analysis_view/
  model/
    filter_spec.py           # FilterSpec dataclass
    view_table_formatting.py # format_cell_for_table, etc.
    pandas_table_model.py    # PandasTableModel
    view_filter_proxy_model.py # ViewSortFilterProxyModel
    view_display_controller.py # shared coordinator

```

```

analysis_view_data_manager.py # existing + row filter helpers
ui/
  tabular_frame.py           # QTableView host
  table_display_toolbar.py    # units + decimals
  filterable_header_view.py   # Phase 4
  column_filter_popup.py      # Phase 4
  graphical_frame.py          # plot from visible df
  analysis_view.py            # orchestrator
  view_sidebar.py             # unchanged (column selection)

```

6. Phased implementation plan (items 1–55)

Numbered steps below replace checklist boxes. Each step maps to the function map in §3. When implementing, ask for full code by **phase and item number** (e.g. Phase 1, item 15).

Phase 0 — Prerequisites (no UI change)

1. Add `FilterSpec` dataclass in `filter_spec.py`
2. Add `AnalysisView.row_filters: list[FilterSpec]` to `project/models.py`
3. Create `view_table_formatting.py` with `format_normalised_cell_value`, `apply_decimal_formatting`, `format_cell_for_table`
4. Verify formatting output matches current `create_dataframe_table` for a sample df (manual or quick test)
5. Fix `self.graphical_frame` reference in `analysis_view.py` (store on widget, pass `view` + `project` to `GraphicalFrame`)

Exit criteria: Formatting helpers exist; model field ready for persistence; graphical frame reachable from orchestrator.

Phase 1 — Model + table view (no row filters yet)

6. Implement `PandasTableModel` class in `pandas_table_model.py` (including `data()`, `rowCount`, `columnCount`, `headerData`)
7. Implement `PandasTableModel.set_dataframe(df, column_specs, project, decimal_settings)`
8. Implement `PandasTableModel.set_column_unit(col, unit)`
9. Implement `PandasTableModel.set_decimal_settings(...)`
10. Implement `ViewDisplayController` class in `view_display_controller.py` (source model only; proxy stub or pass-through)
11. Implement `ViewDisplayController.set_view_data(df, column_specs)`
12. Implement `ViewDisplayController.refresh_formatting()`
13. Create `table_display_toolbar.py` with `_build_display_toolbar()` (units combos + decimal controls)
14. Implement `_sync_units_toolbar_from_specs()` in toolbar or `tabular_frame.py`
15. Refactor `TabularFrame._build_table_view_once()` with `QTableView` attached to controller proxy
16. Implement `TabularFrame.bind_display_controller(controller)`
17. Implement `TabularFrame.set_view_data(df, column_specs)` (replaces `update_table()` for data changes)

18. Implement `TabularFrame._on_toolbar_unit_changed(col, unit)` (emit `column_unit_change` + `controller.refresh_formatting()`)
19. Implement `TabularFrame._on_decimal_settings_changed()`
20. Remove `TabularFrame._create_table`, `update_table`, `_update_table_units_combo_from_specs`, `_link_table_units_combo_with_signal`
21. Implement `AnalysisViewWidget._init_display_controller()`
22. Wire `on_view_df_change` → `controller.set_view_data` in `analysis_view.py`
23. Wire `_on_column_unit_change` → `controller.refresh_formatting()` in `analysis_view.py`
24. Enable `QTableView.setSortingEnabled(True)` on proxy
25. Confirm column tick/untick, unit change, and decimal rounding still work
26. Confirm `mark_modified()` still fires on column and unit changes

Exit criteria: Analysis view table uses `QTableView` + model; no full widget rebuild on unit/decimal change; column selection still updates structure via `modelReset`.

Replaces: Analysis-view usage of `create_table_view_frame` in `tabular_frame.py`.

Phase 2 — Row filter proxy (backend only)

27. Implement `row_filter_mask(df, filter_specs, column_specs)` in `analysis_view_data_manager.py`
28. Implement `on_row_filter_change(view, filter_specs)` in `analysis_view_data_manager.py`
29. Implement `get_visible_row_indices(proxy)` in `analysis_view_data_manager.py`
30. Implement `get_visible_dataframe(view, proxy)` in `analysis_view_data_manager.py`
31. Implement `ViewSortFilterProxyModel` class with `filterAcceptsRow` using `row_filter_mask` in `view_filter_proxy_model.py`
32. Implement `ViewSortFilterProxyModel.set_filters(filter_specs)`
33. Implement `ViewSortFilterProxyModel.source_row_at(proxy_row)`
34. Wire proxy into `ViewDisplayController.set_filters` and `ViewDisplayController.get_visible_dataframe`
35. Implement `TabularFrame.set_row_filters(filter_specs)`
36. Add temporary dev UI (e.g. simple filter line or test button) to verify proxy filtering without Excel popup
37. Verify sort + filter compose correctly (sort visible subset)

Exit criteria: Programmatic filters hide rows without rebuild; visible row count matches pandas mask.

Phase 3 — Graphical frame sync

38. Implement `get_plot_series(view, visible_df)` in `analysis_view_data_manager.py`
39. Implement `GraphicalFrame.bind_display_controller(controller)`
40. Implement `GraphicalFrame.update_from_visible_data(visible_df, column_specs)`
41. Implement `GraphicalFrame._render_plot(visible_df)` (minimal plot: depth vs formation pressure)
42. Implement `GraphicalFrame._clear_plot()`
43. Implement `AnalysisViewWidget._on_display_changed()` and connect `controller.display_changed`
44. Verify: table row filter reduces plotted points
45. Verify: column selection updates both table and plot

Exit criteria: Tabular and graphical views show the same visible row set.

Phase 4 — Excel-style filter UI

- 46. Implement `FilterableHeaderView` in `filterable_header_view.py`
- 47. Implement `ColumnFilterPopup` in `column_filter_popup.py`
- 48. Implement `TabularFrame._on_filter_applied(col, filter_spec)`
- 49. Implement `AnalysisViewWidget._on_row_filter_change()` (calls `on_row_filter_change` + `controller.set_filters`)
- 50. Persist `view.row_filters` on apply; restore on view open
- 51. Call `mark_modified()` on filter change

Exit criteria: User can filter columns from header UI; filters survive save/load.

Phase 5 — Optional cleanup

- 52. Refactor `create_dataframe_table` in `global_functions.py` to call `format_cell_for_table` (DRY with model path)
- 53. Trim unused re-exports in `ui/analysis_view/__init__.py` if desired
- 54. Remove dead backup files (`analysis_view_old.py`, `tabular_frame_old.py`) if still present
- 55. Performance check on 1000+ row dataset (filter, sort, unit change)

Exit criteria: No duplicated display logic; acceptable performance on large tables.

Plan item index (all 55 numbered steps)

Phase	Items	\$3 function map
0	1–5	A (formatting), B (<code>FilterSpec</code> , model field), G (graphical_frame fix)
1	6–26	A, C (<code>PandasTableModel</code>), D (<code>ViewDisplayController</code>), E (tabular + toolbar), H (orchestrator)
2	27–37	B (filter helpers), C (<code>ViewSortFilterProxyModel</code>), D, E (<code>set_row_filters</code>)
3	38–45	B (<code>get_plot_series</code>), G, H (<code>_on_display_changed</code>)
4	46–51	F (filter UI), E, H (<code>_on_row_filter_change</code>)
5	52–55	Optional DRY and housekeeping

7. Summary counts

Layer	New items
Formatting	3 functions
Data manager + types	1 type + 5 functions + 1 model field
Qt model/proxy	2 classes, ~6 methods

Layer	New items
Display controller	1 class, ~5 methods
Tabular UI	~8 functions; 4 removed
Filter UI (Phase 4)	2 classes + 1 handler
Graphical	4 functions
Orchestrator	4 functions (2 new, 2 extended)

Total: ~35 named units (functions + class methods + types), expanded to **55 numbered implementation steps** in §6 (includes verification steps and class methods listed separately).

8. Related documents

- `Design Assets/260623 Connect Column Selection Change Plan KD.md` — column selection workflow (completed)
 - `Design Assets/Analysis_Module_Blueprint.md` — broader analysis module design
-

9. References

- [PyQt6 Model/View sort and filter tutorial](#)
 - [QSortFilterProxyModel documentation](#)
 - [Excel-like filter on QTableWidgetItem \(Stack Overflow\)](#) — header popup pattern for Phase 4
-

10. Printing / PDF export

Tables keep standard markdown pipe format. PDF export uses **portrait A4**, readable fonts (11pt body, 9–9.5pt tables), and `Design Assets/markdown-print.css`.

Cursor / VS Code — Markdown PDF extension

1. Workspace `.vscode/settings.json` enables portrait + print CSS.
2. **Developer: Reload Window** (once after CSS changes).
3. Open the `.md` file → **Ctrl+Shift+P** → **Markdown PDF: Export (pdf)** (or **Export (html)**).
4. Overwrite the previous PDF/HTML in `Design Assets/`.

Pandoc (alternative)

```
cd "C:\Users\KonstantinDanos\OneDrive - KD CyPRES Energy ltd\Constantinos
Danos\Career\Training\Coding\Python\RFT 2026"
pandoc "Design Assets/260630 QTableView Refactor Plan KD.md" -o "Design Assets/260630 QTableView
Refactor Plan KD.pdf" --css="Design Assets/markdown-print.css" -V geometry:margin=14mm
```

Print CSS behaviour

- Portrait A4; full page width (overrides VS Code preview max-width)
- Body 11pt; tables 9.5pt (9pt for 4-column §3 tables); code 9pt
- Table rows may break across pages (avoids large white gaps)
- `table-layout: fixed` + word-wrap on cells

- Inline code: light gray `#e8eaeed`; code blocks: `#f3f4f6`